

A 3.3.3 Updating Data in Databases with SQL



A 3.3.3 Explain how SQL can be used to update data in a database.

SQL can modify databases via INSERT, UPDATE, and DELETE. Indexing impacts update speed and large changes may require index rebuilds.

Updating Data in Databases with SQL

In this section we will explore how Structured Query Language (SQL) is used to modify the data stored within relational databases. Understanding how to update data is fundamental to interacting with and maintaining any dynamic database system. We will cover the three primary SQL commands for data manipulation: **INSERT INTO** for adding new records, **UPDATE SET** for modifying existing records, and **DELETE** for removing records. Finally, we will discuss the crucial performance implications of updating data within indexed columns.

Inserting New Records: The **INSERT INTO** Statement

The **INSERT INTO** statement is used to add new rows, or records, into a table within your database. This is how new information is entered and stored. There are two main ways to use the **INSERT INTO** statement:

1. Inserting into all columns:

If you want to add data to every column in a table, you can use the following basic syntax:

```
INSERT INTO table_name
VALUES (value1, value2, value3, ...);
```

Here:

- **INSERT INTO** is the keyword that initiates the insertion operation.
- **table_name** is the name of the table into which you want to insert the new record.

- **VALUES** is a keyword followed by a comma-separated list of the values for each column in the table. The order of these values **must** match the order of the columns as defined in the table schema.
- Each **value** corresponds to a specific column. The data type of each value must be compatible with the data type defined for its respective column.

Example:

Consider a table named **Students** with the following columns: **StudentID** (INT), **FirstName** (VARCHAR), **LastName** (VARCHAR), and **DateOfBirth** (DATE). To insert a new student record, you might write:

```
INSERT INTO Students  
VALUES (101, 'Alice', 'Smith', '2005-03-15');
```

This statement adds a new row to the **Students** table with the specified values for **StudentID**, **FirstName**, **LastName**, and **DateOfBirth** in that exact order.

2. Inserting into specific columns:

You can also choose to insert data into only certain columns of a table. This is useful when you don't have values for all columns (perhaps some columns allow **NULL** values or have default constraints) or when you only want to populate a subset of the available attributes. The syntax for this is:

```
INSERT INTO table_name (column1, column2, column3, ...)  
VALUES (value1, value2, value3, ...);
```

Here:

- After **table_name**, you specify a comma-separated list of the columns you are providing values for, enclosed in parentheses.
- The **VALUES** clause then contains a comma-separated list of the corresponding values, and their order **must** match the order of the columns listed in the parentheses.

Example:

Using the same **Students** table, if you only wanted to insert the first and last names of a new student, you could write:

```
INSERT INTO Students (FirstName, LastName)
VALUES ('Bob', 'Johnson');
```

In this case, the **StudentID** and **DateOfBirth** columns would either receive their default values (if defined) or be set to **NULL** if no default is specified and **NULL** values are allowed for those columns.

Modifying Existing Data: The **UPDATE SET** Statement

The **UPDATE** statement is used to modify the data in one or more existing records within a table. It allows you to change the values of specific columns for rows that meet a certain condition. The basic syntax of the **UPDATE** statement is:

```
UPDATE table_name
SET column1 = value1, column2 = value2, ...
WHERE condition;
```

Here:

- **UPDATE** is the keyword that initiates the update operation.
- **table_name** is the name of the table you want to modify.
- **SET** is a keyword that indicates which columns you want to update and the new values they should have. You can update multiple columns simultaneously by separating the assignments with commas.
- **column1 = value1** assigns **value1** to **column1**. The new value must be compatible with the column's data type.
- **WHERE condition** is a crucial clause that specifies which rows should be updated. The **condition** is a logical expression that evaluates to true for the rows you want to modify.

Crucial Note on the **WHERE Clause:** If you omit the **WHERE** clause, the **UPDATE** statement will modify **all** rows in the table. This is often unintended and can have significant consequences for your data. Therefore, always ensure you have a well-defined **WHERE** clause when you want to update specific records.

Example:

Suppose Alice Smith from our **Students** table had a date of birth entered incorrectly and it should be '2005-04-20'. We can correct this using the **UPDATE** statement:

```
UPDATE Students
SET DateOfBirth = '2005-04-20'
WHERE FirstName = 'Alice' AND LastName = 'Smith';
```

This statement finds the row where **FirstName** is 'Alice' and **LastName** is 'Smith' and then changes the value in the **DateOfBirth** column to '2005-04-20' for that specific row.

Updating Multiple Columns:

You can update multiple columns in a single **UPDATE** statement by including multiple assignments in the **SET** clause, separated by commas.

Example:

If Bob Johnson's **StudentID** was also missing and should be 102, and we wanted to update both his **StudentID** and **LastName**, we could use:

```
UPDATE Students
SET StudentID = 102, LastName = 'Jones'
WHERE FirstName = 'Bob';
```

This would update both the **StudentID** and **LastName** for the student with the **FirstName** 'Bob'.

Removing Data: The **DELETE** Statement

The **DELETE** statement is used to remove one or more existing records from a table. Like the **UPDATE** statement, it's essential to use a **WHERE** clause to specify which rows you want to delete to avoid unintentionally removing all data from a table. The basic syntax of the **DELETE** statement is:

```
DELETE FROM table_name
WHERE condition;
```

Here:

- **DELETE FROM** are the keywords that initiate the deletion operation.
- **table_name** is the name of the table from which you want to remove records.

- **WHERE condition** is the clause that specifies which rows should be deleted. Only rows for which the **condition** evaluates to true will be removed.

Critical Warning Regarding the WHERE Clause: Similar to the **UPDATE** statement, if you omit the **WHERE** clause in a **DELETE** statement, **all** rows in the specified table will be deleted. Use this with extreme caution and only when you genuinely intend to empty the entire table.

Example:

Suppose a student with **StudentID** 101 has left the school, and we need to remove their record from the **Students** table:

```
DELETE FROM Students  
WHERE StudentID = 101;
```

This statement will find the row where **StudentID** is 101 and remove that entire record from the **Students** table.

Deleting All Rows (Use with Caution):

To delete all rows from a table while keeping the table structure intact, you can use the **DELETE** statement without a **WHERE** clause:

```
DELETE FROM Students;
```

Note: While **DELETE** removes rows, it doesn't reset any auto-increment counters associated with the table. If you need to remove all rows and reset the counter, you might consider using the **TRUNCATE TABLE** command (though its behaviour can vary slightly between database systems and might not be part of the standard SQL covered in detail in this course).

Performance Implications of Updating Indexed Columns

Database indexes are special lookup tables that the database search engine can use to speed up data retrieval. They work similarly to the index in a book, allowing the database to quickly locate specific rows without having to scan through the entire table. Indexes are created on one or more columns of a table.

When you update data in a column that is part of an index, the database not only needs to modify the data in the table itself but also needs to update the index to reflect this change. This additional

work can have performance implications, especially in tables with a large number of rows or when performing frequent updates on indexed columns.

Here's a breakdown of the performance considerations:

- **Increased Write Overhead:** Every time an indexed column is updated, the database must also update the index structure. This involves locating the old value in the index, removing it, and inserting the new value in the correct position within the index. For tables with many indexes on the columns being updated, this overhead can become significant, slowing down the **UPDATE** operation.
- **Index Rebuilding or Reorganisation:** Over time, and especially after a large number of updates (or inserts and deletes), the index structure can become fragmented or inefficient. This means that the logical ordering of the index might no longer closely match its physical storage on disk, leading to slower lookups even for indexed columns.
 - In such cases, the database administrator might need to **rebuild** or **reorganise** the index. Rebuilding typically involves creating a new copy of the index, which can be resource-intensive but often results in a more optimised index. Reorganising is a less intensive process that attempts to defragment the existing index structure.
- **Impact on Other Operations:** While updating an indexed column can slow down the update itself, the presence of the index will still generally speed up **SELECT** queries that use the indexed column in their **WHERE** clause. Therefore, there's often a trade-off between the performance of read (SELECT) and write (INSERT, UPDATE, DELETE) operations on indexed columns.

Considerations for Significant Data Modifications:

If you perform a large number of updates on indexed columns, you might observe a degradation in the performance of queries that rely on those indexes. In such scenarios, database administrators might consider the following:

- **Dropping and Recreating Indexes:** For very large bulk updates, it might be more efficient to drop the relevant indexes before performing the updates and then recreate them afterwards. This avoids the overhead of updating the index for each individual row modification. However, remember that dropping an index will negatively impact the performance of **SELECT** queries that rely on it until it is recreated.
- **Index Maintenance Schedules:** Modern database management systems often have features for automatic or scheduled index maintenance, including defragmentation or rebuilding, to ensure optimal performance over time.
- **Careful Index Design:** The key is to design indexes thoughtfully, considering the columns that are frequently used in **WHERE** clauses for **SELECT** queries and balancing this with the potential overhead on write operations. Avoid indexing columns that are very frequently updated and rarely used in search conditions.

Indexes are crucial for improving the speed of data retrieval, but be aware that updating indexed columns comes with a performance cost. Significant data modifications on indexed

columns might necessitate index rebuilding or reorganisation to maintain optimal database performance.

All materials on this website are for the exclusive use of teachers and students at subscribing schools for the period of their subscription. Any unauthorised copying or posting of materials on other websites is an infringement of our copyright and could result in your account being blocked and legal action being taken against you.

 **Report a problem**